



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Introduction to Data Science

Mid-Term Project Report

Summer 2025 - 2026

Section: B

Group: 04

Project Contributions

SL	Student Name	ID	Contributions
1.	Md. Asraf Uddin	23-52310-2	1, 2, 8, 14
2.	Farhan Chowdhury	23-52314-2	4, 5, 6, 11
3.	Md. Shihab Uddin	23-54470-3	3, 7, 10, 15
4.	Md. Shahriyar Hasan	23-52515-2	9, 12, 13

Table Of Content

Contents	Page No.
Dataset Description	03
Project Implementation Details	03
Task 1: Exploratory Data Analysis	03 - 04
Task 2: Handling Missing Values	05
Task 3: Outlier Detection and Handling	06
Task 4: Attribute Type Conversion	07
Task 5: Normalization	07 – 08
Task 6: Duplicate Detection and Removal	08
Task 7: Data Filtering	09
Task 8: Invalid Data Detection and Handling	10 – 11
Task 9: Feature Engineering	11
Task 10: Class Balancing with Under- and Over-Sampling	12
Task 11: Training and Testing Split	13
Task 12: Mean vs Median Imputation Comparison	14
Task 13: Descriptive Statistics by Target Class	15
Task 14: Comparing Averages Across Two Categories	16
Task 15: Variability Comparison Across Categories	17
Project Code	18 - 22

Dataset Description

The Sleep Health Dataset contains records of 100,000 individuals with 32 attributes describing demographic, lifestyle, sleep, and health-related features. Attributes include age, gender, occupation, BMI, country, sleep duration (hours), sleep quality score, REM percentage, deep sleep percentage, sleep latency, wake episodes, caffeine and alcohol intake before bed, screen time, exercise, daily steps, nap duration, stress score, work hours, chronotype, mental health condition, resting heart rate, sleep aid use, shift work status, room temperature, weekend sleep difference, season, day type, and cognitive performance score.

The target variable is `sleep_disorder_risk`, a categorical attribute with four classes: Healthy, Mild, Moderate, and Severe. The dataset contains both numerical attributes (e.g., age, bmi, sleep_duration_hrs, stress_score) and categorical attributes (e.g., gender, chronotype, mental_health_condition), which makes it suitable for the requirements of this project. To fulfill the project requirements, the dataset was modified by introducing missing values in selected columns, duplicate records, and invalid entries such as negative ages and impossible sleep durations.

Project Implementation Detail

Task 1: Exploratory Data Analysis

Description: Perform an initial exploratory analysis of the dataset by examining its dimensions, attribute types, summary statistics, and target variable distribution using built-in R functions.

CODE

```
1 cat("\n Task1: Exploratory Data Analysis \n")
2 data <- read.csv("C:\\Users\\Farhan\\Downloads\\archive\\sleep_health_dataset_modified.csv",
3                 header = TRUE, sep = ",",
4                 stringsAsFactors = FALSE, na.strings = c("NA", ""))
5
6 raw_data <- data
7
8 cat("Structure of the dataset: \n")
9 str(data)
10
11 cat("\n First 6 rows: \n")
12 print(head(data))
13
14 cat("\n Dataset size: \n")
15 cat("Rows:", nrow(data), "\n")
16 cat("Columns:", ncol(data), "\n")
17
18 cat("\nColumn names: \n")
19 print(names(data))
20
21 cat("\n Summary of the dataset: \n")
22 print(summary(data))
23
24 cat("\n Target variable (sleep_disorder_risk) distribution: \n")
25 print(table(data$sleep_disorder_risk))
26 cat("Proportions (%): \n")
27 print(round(prop.table(table(data$sleep_disorder_risk)) * 100, 2))
28
```

OUTPUT

```
Structure of the dataset:
> str(data)
'data.frame': 100000 obs. of 32 variables:
 $ person_id      : int  1 2 3 4 5 6 7 8 9 10 ...
 $ age            : int  29 55 42 37 23 23 20 48 37 41 ...
 $ gender         : chr  "Female" "Female" "Male" "Female" ...
 $ occupation     : chr  "Driver" "Software Engineer" "Nurse" "Student" ...
 $ bmi            : chr  "25.7" "22" "25" "29.5" ...
 $ country        : chr  "Japan" "USA" "India" "India" ...
 $ sleep_duration_hrs : int  6.19 "8.32" "3.74" "2" ...
 $ sleep_quality_score : num  6.6 6.9 1 6.4 3.2 5.7 5.1 6.5 5.4 4.9 ...
 $ rem_percentage  : num  22.5 26.9 20.2 17.7 23.3 17.3 17.5 25.3 18.9 21.4 ...
 $ deep_sleep_percentage : num  19.3 14.9 16.2 17.7 18.3 20.1 18.8 22.3 15.7 15.5 ...
 $ sleep_latency_mins : int  16 17 26 13 30 29 31 22 37 24 ...
 $ wake_episodes_per_night : int  3 4 4 5 2 2 4 0 3 ...
 $ caffeine_mg_before_bed : int  0 0 0 0 40 150 0 150 400 0 ...
 $ alcohol_units_before_bed : int  0 0 2 1 0 2 3 0 0 0 ...
 $ screen_time_before_bed_mins : int  32 33 89 52 72 72 148 59 40 88 ...
 $ exercise_day    : int  0 1 1 0 0 1 1 0 0 ...
 $ steps_that_day  : int  6592 10111 9222 9190 4273 9170 10786 6194 6083 5207 ...
 $ nap_duration_mins : int  0 8 28 40 0 0 0 46 44 0 ...
 $ stress_score     : num  4.4 4.7 8.4 4.9 7.4 5.5 5.5 6 5.7 ...
 $ work_hours_that_day : num  10.7 3.3 6.6 7.0 10.4 6.2 4.3 2.8 5.4 12.7 ...
 $ chronotype      : chr  "Morning" "Neutral" "Neutral" "Morning" ...
 $ mental_health_condition : chr  "Healthy" "Healthy" "Both" "Healthy" ...
 $ heart_rate_resting_bpm : int  63 52 72 71 71 71 79 60 74 80 ...
 $ sleep_aid_used   : int  0 1 0 0 0 1 0 1 0 ...
 $ shift_work      : int  0 0 1 0 0 0 0 0 0 ...
 $ room_temperature_celsius : num  20.1 18 17.9 19.1 19.7 22.7 17.6 21.6 17.8 24.4 ...
 $ weekend_sleep_diff_hrs : num  1.84 0.13 1.67 2.37 1.26 0.7 0.73 0.47 1.8 1.51 ...
 $ season          : chr  "Autumn" "Winter" "Spring" "Summer" ...
 $ day_type        : chr  "Weekday" "Weekend" "Weekend" "Weekend" ...
 $ cognitive_performance_score : num  73.4 99.4 2.5 67.8 38.1 NA 57.9 NA 53.6 67.8 ...
 $ sleep_disorder_risk : chr  "Healthy" "Healthy" "Severe" "Healthy" ...
 $ felt_rested     : int  0 1 0 0 0 1 1 0 0 1 ...

>
> cat("\n Dataset size: \n")
Dataset size:
> cat("Rows:", nrow(data), "\n")
Rows: 100000
> cat("Columns:", ncol(data), "\n")
Columns: 32
> cat("\nColumn names: \n")
Column names:
> print(names(data))
[1] "person_id"      "age"          "gender"
[4] "occupation"     "bmi"          "country"
[7] "sleep_duration_hrs" "sleep_quality_score" "rem_percentage"
[10] "deep_sleep_percentage" "sleep_latency_mins" "wake_episodes_per_night"
[13] "caffeine_mg_before_bed" "alcohol_units_before_bed" "screen_time_before_bed_mins"
[16] "exercise_day"    "steps_that_day"    "nap_duration_mins"
[19] "stress_score"    "work_hours_that_day" "chronotype"
[22] "mental_health_condition" "heart_rate_resting_bpm" "sleep_aid_used"
[25] "shift_work"      "room_temperature_celsius" "weekend_sleep_diff_hrs"
[28] "season"          "day_type"        "cognitive_performance_score"
[31] "sleep_disorder_risk" "felt_rested"

> cat("\n Summary of the dataset: \n")

Summary of the dataset:
> print(summary(data))
  person_id   age      gender      occupation      bmi
Min.   : 1   Min.   :-15.00   Length   :100000   Length   :100000   Length   :100000
1st Qu.:25001 1st Qu.: 25.00   N.unique   : 6   N.unique   : 12   N.unique   : 474
Median :50001 Median : 33.00   N.blank    : 0   N.blank    : 0   N.blank    : 0
Mean   :50001 Mean   : 33.82   Min.nchar  : 1   Min.nchar  : 5   Min.nchar  : 1
3rd Qu.:75000 3rd Qu.: 42.00   Max.nchar  : 7   Max.nchar  : 17   Max.nchar  : 5
Max.   :100000 Max.   : 69.00           NAS      : 5058   NAS      : 8043

  country      sleep_duration_hrs sleep_quality_score rem_percentage deep_sleep_percentage
Length   :100000 Length   :100000 Min.   : 1.000 Min.   :10.00 Min.   : 5.00
N.unique : 15   N.unique : 1200 1st Qu.: 3.800 1st Qu.:18.00 1st Qu.:17.40
N.blank  : 0   N.blank  : 0 Median : 4.900 Median :20.30 Median :20.30
Min.nchar: 2   Min.nchar: 1 Mean   : 4.871 Mean :20.24 Mean :20.25
Max.nchar:11   Max.nchar: 6 3rd Qu.: 6.000 3rd Qu.:22.60 3rd Qu.:23.20
NAS       : 8094 Max.   :10.000 Max.   :30.00 Max.   :30.00

  sleep_latency_mins wake_episodes_per_night caffeine_mg_before_bed alcohol_units_before_bed
Min.   : 1.00   Min.   :0.000 Min.   : 0.00 Min.   :0.0000
1st Qu.:14.00 1st Qu.:2.000 1st Qu.: 0.00 1st Qu.:0.0000
Median :19.00 Median :3.000 Median : 0.00 Median :0.0000
Mean   :19.84 Mean   :3.346 Mean :38.85 Mean :0.5968
3rd Qu.:25.00 3rd Qu.:5.000 3rd Qu.: 80.00 3rd Qu.:1.0000
Max.   :58.00 Max.   :8.000 Max.   :400.00 Max.   :6.0000

sleep_aid_used shift_work room_temperature_celsius weekend_sleep_diff_hrs
Min.   :0.0000 Min.   :0.00000 Min.   :15.00 Min.   : -1.000
1st Qu.:0.0000 1st Qu.:0.00000 1st Qu.:18.50 1st Qu.: 0.670
Median :1.0000 Median :0.00000 Median :20.50 Median : 1.200
Mean   :0.5616 Mean   :0.08442 Mean :20.54 Mean   : 1.201
3rd Qu.:1.0000 3rd Qu.:0.00000 3rd Qu.:22.50 3rd Qu.: 1.740
Max.   :1.0000 Max.   :1.00000 Max.   :28.00 Max.   : 3.000

  season      day_type      cognitive_performance_score sleep_disorder_risk
Length   :100000 Length   :100000 Min.   : 0.00 Length   :100000
N.unique : 4   N.unique : 2 1st Qu.: 44.00 N.unique : 4
N.blank  : 0   N.blank  : 0 Median : 60.60 N.blank  : 0
Min.nchar: 6   Min.nchar: 7 Mean   :59.55 Min.nchar: 4
Max.nchar: 6   Max.nchar: 7 3rd Qu.:76.40 Max.nchar: 8
Max.   :110.00
NAS      :7881

  felt_rested
Min.   :0.0000
1st Qu.:0.0000
Median :0.0000
Mean   :0.3902
3rd Qu.:1.0000
Max.   :1.0000

>
> cat("\n Target variable (sleep_disorder_risk) distribution: \n")
Target variable (sleep_disorder_risk) distribution:
> print(table(data$sleep_disorder_risk))
Healthy Mild Moderate Severe
54159 33477 8298 4066
> cat("Proportions (%): \n")
Proportions (%):
> print(round(prop.table(table(data$sleep_disorder_risk)) * 100, 2))
Healthy Mild Moderate Severe
54.16 33.48 8.30 4.07
>
```

Output: The output displays the dataset structure, first six records, number of rows and columns, column names, summary statistics, and the distribution of the target variable. It shows that the dataset contains both numerical and categorical attributes.

Task 2: Handling Missing Values

Description: Count the missing values in each column, replace missing values in numerical attributes with the median, replace missing values in categorical attributes with the mode, and verify the result.

CODE

```
78
79 cat("\n===== Task 2: Missing Value Treatment =====\n")
80 cat("Missing values BEFORE treatment:\n")
81 print(colSums(is.na(data)))
82 cat("Total missing values:", sum(is.na(data)), "\n\n")
83
84 cat("--- Imputing numerical columns with median ---\n")
85 data$bmi[is.na(data$bmi)] <- median(data$bmi, na.rm = TRUE)
86 cat("bmi imputed with median\n")
87
88 data$sleep_duration_hrs[is.na(data$sleep_duration_hrs)] <- median(data$sleep_duration_hrs, na.rm = TRUE)
89 cat("sleep_duration_hrs imputed with median\n")
90
91 data$cognitive_performance_score[is.na(data$cognitive_performance_score)] <- median(data$cognitive_performance_score, na.rm = TRUE)
92 cat("cognitive_performance_score imputed with median\n")
93
94 cat("\n--- Imputing categorical columns with mode ---\n")
95 get_mode <- function(v) {
96   uniqv <- unique(v[!is.na(v)])
97   return(uniqv[which.max(tabulate(match(v, uniqv)))] )
98 }
99
100 data$occupation[is.na(data$occupation)] <- get_mode(data$occupation)
101 cat("occupation imputed with mode:", get_mode(raw_data$occupation), "\n")
102
103 data$mental_health_condition[is.na(data$mental_health_condition)] <- get_mode(data$mental_health_condition)
104 cat("mental_health_condition imputed with mode:", get_mode(raw_data$mental_health_condition), "\n")
105
106 cat("\nMissing values AFTER treatment (stress_score handled in Task 12):\n")
107 print(colSums(is.na(data)))
108
```

OUTPUT

```
Task2: Missing Value Treatment
> cat("Missing values BEFORE treatment:\n")
Missing values BEFORE treatment:
> print(colSums(is.na(data)))
  person_id      age      gender
         0         0           0
  occupation      bmi      country
    5058      9038           0
  sleep_duration_hrs  sleep_quality_score  rem_percentage
    10118              0              0
  deep_sleep_percentage  sleep_latency_mins  wake_episodes_per_night
         0              0              0
  caffeine_mg_before_bed  alcohol_units_before_bed  screen_time_before_bed_mins
         0              0              0
  exercise_day      steps_that_day  nap_duration_mins
         0              0              0
  stress_score      work_hours_that_day  chronotype
    8109              0              0
  mental_health_condition  heart_rate_resting_bpm  sleep_aid_used
    5075              0              0
  shift_work      room_temperature_celsius  weekend_sleep_diff_hrs
         0              0              0
  season      day_type  cognitive_performance_score
         0              0              7881
  sleep_disorder_risk  felt_rested
         0              0
> cat("Total missing values:", sum(is.na(data)), "\n\n")
Total missing values: 45279
> |
```

```
Missing values AFTER treatment (stress_score handled in Task 12):
> print(colSums(is.na(data)))
  person_id      age      gender
         0         0           0
  occupation      bmi      country
         0         0           0
  sleep_duration_hrs  sleep_quality_score  rem_percentage
         0              0              0
  deep_sleep_percentage  sleep_latency_mins  wake_episodes_per_night
         0              0              0
  caffeine_mg_before_bed  alcohol_units_before_bed  screen_time_before_bed_mins
         0              0              0
  exercise_day      steps_that_day  nap_duration_mins
         0              0              0
  stress_score      work_hours_that_day  chronotype
    8109              0              0
  mental_health_condition  heart_rate_resting_bpm  sleep_aid_used
         0              0              0
  shift_work      room_temperature_celsius  weekend_sleep_diff_hrs
         0              0              0
  season      day_type  cognitive_performance_score
         0              0              0
  sleep_disorder_risk  felt_rested
         0              0
> cat("Total missing values:", sum(is.na(data)), "\n\n")
Total missing values: 8109
> |
```

Output Description: The output first shows the number of missing values before treatment. After applying median and mode imputation, the missing-value count is displayed again, confirming that the selected columns were successfully cleaned.

Task 3: Outlier Detection and Handling

Description: Create functions to detect outliers using the IQR method and replace extreme values with the lower or upper IQR limits. Apply the method to selected numerical attributes.

CODE

```
109
110 cat("\n Task3: Outlier Detection and Handling \n")
111 detect_outliers_iqr <- function(x) {
112   q1 <- quantile(x, 0.25, na.rm = TRUE)
113   q3 <- quantile(x, 0.75, na.rm = TRUE)
114   iqr_val <- q3 - q1
115   return(which(x < q1 - 1.5 * iqr_val | x > q3 + 1.5 * iqr_val))
116 }
117
118 cap_outliers_iqr <- function(x) {
119   q1 <- quantile(x, 0.25, na.rm = TRUE)
120   q3 <- quantile(x, 0.75, na.rm = TRUE)
121   iqr_val <- q3 - q1
122   lower <- q1 - 1.5 * iqr_val
123   upper <- q3 + 1.5 * iqr_val
124   x[x < lower] <- lower
125   x[x > upper] <- upper
126   return(x)
127 }
128
129 cat(" steps_that_day \n")
130 outliers_steps <- detect_outliers_iqr(data$steps_that_day)
131 cat("Outliers before capping:", length(outliers_steps), "\n")
132 data$steps_that_day <- cap_outliers_iqr(data$steps_that_day)
133 cat("Outliers after capping:", length(detect_outliers_iqr(data$steps_that_day)), "\n")
134
135 cat("\n screen_time_before_bed_mins \n")
136 outliers_screen <- detect_outliers_iqr(data$screen_time_before_bed_mins)
137 cat("Outliers before capping:", length(outliers_screen), "\n")
138 data$screen_time_before_bed_mins <- cap_outliers_iqr(data$screen_time_before_bed_mins)
139 cat("Outliers after capping:", length(detect_outliers_iqr(data$screen_time_before_bed_mins)), "\n")
140
```

OUTPUT

```
Task3: Outlier Detection and Handling
> cat(" steps_that_day \n")
steps_that_day
> outliers_steps <- detect_outliers_iqr(data$steps_that_day)
> cat("Outliers before capping:", length(outliers_steps), "\n")
Outliers before capping: 272
> data$steps_that_day <- cap_outliers_iqr(data$steps_that_day)
> cat("Outliers after capping:", length(detect_outliers_iqr(data$steps_that_day)), "\n")
Outliers after capping: 0
>
> cat("\n screen_time_before_bed_mins \n")

screen_time_before_bed_mins
> outliers_screen <- detect_outliers_iqr(data$screen_time_before_bed_mins)
> cat("Outliers before capping:", length(outliers_screen), "\n")
Outliers before capping: 5781
> data$screen_time_before_bed_mins <- cap_outliers_iqr(data$screen_time_before_bed_mins)
> cat("Outliers after capping:", length(detect_outliers_iqr(data$screen_time_before_bed_mins)), "\n")
Outliers after capping: 0
> |
```

Output Description: The output shows the number of outliers before applying capping and confirms that no outliers remain after the capping process.

Task 4: Attribute Type Conversion

Description: Convert the age attribute into age-group categories using cut() and encode the gender attribute into numeric values using factor encoding.

CODE

```
142
143 cat("\n Task4: Numeric to Categorical and Categorical to Numeric \n")
144 cat(" Numeric to Categorical: age -> age_group \n")
145 data$age_group <- cut(data$age,
146                       breaks = c(17, 30, 50, 100),
147                       labels = c("Young", "Middle-aged", "Senior"))
148 cat("age_group distribution:\n")
149 print(table(data$age_group))
150
151 cat("\n Categorical to Numeric: gender -> gender_num \n")
152 data$gender_num <- as.numeric(factor(data$gender,
153                                     levels = c("Female", "Male", "Other")))
154 cat("Mapping: Female=1, Male=2, Other=3\n")
155 print(table(data$gender, data$gender_num))
156
157
```

OUTPUT

```
Task4: Numeric to Categorical and Categorical to Numeric
> cat(" Numeric to Categorical: age -> age_group \n")
Numeric to Categorical: age -> age_group
> data$age_group <- cut(data$age,
+                       breaks = c(17, 30, 50, 100),
+                       labels = c("Young", "Middle-aged", "Senior"))
> cat("age_group distribution:\n")
age_group distribution:
> print(table(data$age_group))

      Young Middle-aged   Senior
40110      50477      9413

>
> cat("\n Categorical to Numeric: gender -> gender_num \n")

Categorical to Numeric: gender -> gender_num
> data$gender_num <- as.numeric(factor(data$gender,
+                                     levels = c("Female", "Male", "Other")))
> cat("Mapping: Female=1, Male=2, Other=3\n")
Mapping: Female=1, Male=2, Other=3
> print(table(data$gender, data$gender_num))

      1      2      3
Female 49703      0      0
Male   0 47360      0
Other  0      0 2937

> |
```

Output Description: The output displays the distribution of the newly created age groups and the mapping between gender categories and their corresponding numeric values.

Task 5: Normalization

Description: Define a Min-Max Normalization function and apply it to the stress_score attribute to scale the values between 0 and 1.

CODE

```
158
159 cat("\n Task5: Normalization \n")
160 min_max_normalize <- function(x) {
161   return((x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE)))
162 }
163
164 cat("stress_score BEFORE normalization:\n")
165 print(summary(data$stress_score))
166
167 data$stress_score_minmax <- min_max_normalize(data$stress_score)
168 cat("\nstress_score AFTER Min-Max normalization:\n")
169 print(summary(data$stress_score_minmax))
170
```

OUTPUT

```
Task5: Normalization
> min_max_normalize <- function(x) {
+   return((x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE)))
+ }
>
> cat("stress_score BEFORE normalization:\n")
stress_score BEFORE normalization:
> print(summary(data$stress_score))
   Min. 1st Qu.  Median     Mean 3rd Qu.    Max.     NAs
 1.000   4.800   5.900   8.841   6.900 180.000   8109
>
> data$stress_score_minmax <- min_max_normalize(data$stress_score)
> cat("\nstress_score AFTER Min-Max normalization:\n")

stress_score AFTER Min-Max normalization:
> print(summary(data$stress_score_minmax))
   Min. 1st Qu.  Median     Mean 3rd Qu.    Max.     NAs
0.00000 0.02123 0.02737 0.04380 0.03296 1.00000   8109
> |
```

Output Description: The output compares the summary statistics of stress_score before and after normalization, showing that all values have been scaled successfully.

Task 6: Duplicate Detection and Removal

Description: Detect duplicate records after excluding the person_id column, remove the duplicate rows, and display the updated dataset size.

CODE

```
172
173 cat("\n Task 6: Duplicate Detection and Removal \n")
174 dup_count <- sum(duplicated(data[, -which(names(data) == "person_id")]))
175 cat("Total rows:", nrow(data), "\n")
176 cat("Duplicate rows found:", dup_count, "\n")
177 data <- data[!duplicated(data[, -which(names(data) == "person_id")]), ]
178 cat("Rows after removing duplicates:", nrow(data), "\n")
179
180
```

OUTPUT

```
Task 7: Data Filtering
> cat("Filtering data to keep only top 5 countries by frequency\n\n")
Filtering data to keep only top 5 countries by frequency

> cat("Country distribution:\n")
Country distribution:
> print(sort(table(data$country), decreasing = TRUE))

      USA      India      Japan      UK      Brazil      Germany      Canada      France
20050    17839     8089     8023     7145     6026     4995     4982
Australia  4948    Italy South Korea Mexico Netherlands Spain      Sweden
4948      4018      3971      3887      3040      2011      971

>
> top5 <- names(sort(table(data$country), decreasing = TRUE))[1:5]
> cat("\nTop 5 countries:", top5, "\n")

Top 5 countries: USA India Japan UK Brazil
>
> data_filtered <- data[data$country %in% top5, ]
> cat("Rows before filtering:", nrow(data), "\n")
Rows before filtering: 99995
> cat("Rows after filtering:", nrow(data_filtered), "\n")
Rows after filtering: 61146
> |
```

Output Description: The output reports the total number of rows, the number of duplicate rows found, and the total number of rows remaining after removing duplicates.

Task 7: Data Filtering

Description: Count the frequency of each country, identify the top five most frequent countries, and filter the dataset to keep only those records.

CODE

```
183 cat("\n Task 7: Data Filtering \n")
184 cat("Filtering data to keep only top 5 countries by frequency\n\n")
185 cat("Country distribution:\n")
186 print(sort(table(data$country), decreasing = TRUE))
187
188 top5 <- names(sort(table(data$country), decreasing = TRUE))[1:5]
189 cat("\nTop 5 countries:", top5, "\n")
190
191 data_filtered <- data[data$country %in% top5, ]
192 cat("Rows before filtering:", nrow(data), "\n")
193 cat("Rows after filtering:", nrow(data_filtered), "\n")
194
```

OUTPUT

```
Task 7: Data Filtering
> cat("Filtering data to keep only top 5 countries by frequency\n\n")
Filtering data to keep only top 5 countries by frequency

> cat("Country distribution:\n")
Country distribution:
> print(sort(table(data$country), decreasing = TRUE))

      USA      India      Japan      UK      Brazil      Germany      Canada      France
20050    17839    8089    8023    7145    6026    4995    4982
Australia  Italy South Korea  Mexico Netherlands  Spain  Sweden
 4948    4018    3971    3887    3040    2011    971

>
> top5 <- names(sort(table(data$country), decreasing = TRUE))[1:5]
> cat("\nTop 5 countries:", top5, "\n")

Top 5 countries: USA India Japan UK Brazil
>
> data_filtered <- data[data$country %in% top5, ]
> cat("Rows before filtering:", nrow(data), "\n")
Rows before filtering: 99995
> cat("Rows after filtering:", nrow(data_filtered), "\n")
Rows after filtering: 61146
> |
```

Output Description: The output displays the country frequency table, the selected top five countries, and the dataset size before and after filtering.

Task 8: Invalid Data Detection and Handling

Description: Detect invalid values in different attributes, convert incorrect values where necessary, replace invalid values with appropriate values, and clean inconsistent categorical entries.

CODE

```
31 cat("\n Task8: Invalid Data Detection and Handling \n")
32 cat("(Performed early so numeric columns are clean for subsequent tasks) \n\n")
33
34 cat("Cleaning non-numeric values in bmi \n")
35 invalid_bmi_text <- sum(grepl("[^0-9.\\-]", data$bmi), na.rm = TRUE)
36 cat("Non-numeric bmi values found:", invalid_bmi_text, "\n")
37 data$bmi <- suppressWarnings(as.numeric(data$bmi))
38 cat("Converted bmi to numeric (non-numeric values become NA) \n")
39
40 cat("\n Cleaning non-numeric values in sleep_duration_hrs \n")
41 invalid_text_idx <- grepl("[^0-9.\\-]", data$sleep_duration_hrs)
42 cat("Non-numeric sleep_duration_hrs values found:", sum(invalid_text_idx, na.rm = TRUE), "\n")
43 cat("Examples:", head(data$sleep_duration_hrs[invalid_text_idx], 5), "\n")
44 data$sleep_duration_hrs <- suppressWarnings(as.numeric(data$sleep_duration_hrs))
45 cat("Converted to numeric (non-numeric values become NA) \n")
46
47 cat("\n Handling negative age values \n")
48 invalid_age_count <- sum(data$age < 0, na.rm = TRUE)
49 cat("Negative age values found:", invalid_age_count, "\n")
50 data$age[data$age < 0] <- NA
51 med_age <- round(median(data$age, na.rm = TRUE))
52 data$age[is.na(data$age)] <- med_age
53 cat("Replaced with median age:", med_age, "\n")
54
55 cat("\n Handling negative sleep_duration_hrs \n")
56 neg_sleep <- sum(data$sleep_duration_hrs < 0, na.rm = TRUE)
57 cat("Negative sleep_duration values found:", neg_sleep, "\n")
58 data$sleep_duration_hrs[data$sleep_duration_hrs < 0] <- NA
59 cat("Set negative sleep values to NA (will be imputed in Task 2)\n")
60
61 cat("\n Handling heart_rate_resting_bpm <= 30 \n")
62 low_hr <- sum(data$heart_rate_resting_bpm <= 30, na.rm = TRUE)
63 cat("Heart rate <= 30 values found:", low_hr, "\n")
64 data$heart_rate_resting_bpm[data$heart_rate_resting_bpm <= 30] <- NA
65 med_hr <- round(median(data$heart_rate_resting_bpm, na.rm = TRUE))
66 data$heart_rate_resting_bpm[is.na(data$heart_rate_resting_bpm)] <- med_hr
67 cat("Replaced with median heart rate:", med_hr, "\n")
68
69 cat("\n Handling invalid gender values \n")
70 cat("Gender distribution before cleaning:\n")
71 print(table(data$gender))
72 data$gender[!(data$gender %in% c("Male", "Female", "Other"))] <- "Other"
73 cat("Gender distribution after cleaning:\n")
74 print(table(data$gender))
```

OUTPUT

```
> cat("Cleaning non-numeric values in bmi\n")
Cleaning non-numeric values in bmi
> invalid_bmi_text <- sum(grep("[^0-9\\.\\-]", data$bmi), na.rm = TRUE)
> cat("Non-numeric bmi values found:", invalid_bmi_text, "\n")
Non-numeric bmi values found: 995
> data$bmi <- suppressWarnings(as.numeric(data$bmi))
> cat("Converted bmi to numeric (non-numeric values become NA)\n")
Converted bmi to numeric (non-numeric values become NA)
>
> cat("\n Cleaning non-numeric values in sleep_duration_hrs\n")

Cleaning non-numeric values in sleep_duration_hrs
> invalid_text_idx <- grep("[^0-9\\.\\-]", data$sleep_duration_hrs)
> cat("Non-numeric sleep_duration_hrs values found:", sum(invalid_text_idx, na.rm = TRUE), "\n")
Non-numeric sleep_duration_hrs values found: 1042
> cat("Examples:", head(data$sleep_duration_hrs[invalid_text_idx], 5), "\n")
Examples: 5.98a 5.59a 6.26a 7.6a 6.5a
> data$sleep_duration_hrs <- suppressWarnings(as.numeric(data$sleep_duration_hrs))
> cat("Converted to numeric (non-numeric values become NA)\n")
Converted to numeric (non-numeric values become NA)
>
> cat("\n Handling negative age values\n")

Handling negative age values
> invalid_age_count <- sum(data$age < 0, na.rm = TRUE)
> cat("Negative age values found:", invalid_age_count, "\n")
Negative age values found: 2002
> data$age[data$age < 0] <- NA
> med_age <- round(median(data$age, na.rm = TRUE))
> data$age[is.na(data$age)] <- med_age
> cat("Replaced with median age:", med_age, "\n")
Replaced with median age: 33
>
> cat("\n Handling negative sleep_duration_hrs\n")

Handling negative sleep_duration_hrs
> neg_sleep <- sum(data$sleep_duration_hrs < 0, na.rm = TRUE)
> cat("Negative sleep_duration values found:", neg_sleep, "\n")
Negative sleep_duration values found: 982
> data$sleep_duration_hrs[data$sleep_duration_hrs < 0] <- NA
> cat("Set negative sleep values to NA (will be imputed in Task 2)\n")
Set negative sleep values to NA (will be imputed in Task 2)
>
> cat("\n Handling heart_rate_resting_bpm <= 30\n")

Handling heart_rate_resting_bpm <= 30
> low_hr <- sum(data$heart_rate_resting_bpm <= 30, na.rm = TRUE)
> cat("Heart rate <= 30 values found:", low_hr, "\n")
Heart rate <= 30 values found: 639
> data$heart_rate_resting_bpm[data$heart_rate_resting_bpm <= 30] <- NA
> med_hr <- round(median(data$heart_rate_resting_bpm, na.rm = TRUE))
> data$heart_rate_resting_bpm[is.na(data$heart_rate_resting_bpm)] <- med_hr
> cat("Replaced with median heart rate:", med_hr, "\n")
Replaced with median heart rate: 67
>
> cat("\n Handling invalid gender values\n")

Handling invalid gender values
> cat("Gender distribution before cleaning:\n")
Gender distribution before cleaning:
> print(table(data$gender))

Female Male Other Unknown X Z
49703 47360 1950 348 337 302
> data$gender[(data$gender %in% c("Male", "Female", "Other"))] <- "Other"
> cat("Gender distribution after cleaning:\n")
Gender distribution after cleaning:
> print(table(data$gender))

Female Male Other
49703 47360 2937
```

Output Description: The output shows the number of invalid records detected for each attribute and confirms that the invalid values were corrected using suitable cleaning methods.

Task 9: Feature Engineering

Description: Generate a new feature named `sleep_efficiency` by adding the values of `deep_sleep_percentage` and `rem_percentage`, then summarize the new attribute.

CODE

```
197 cat("\n Task9: Feature Engineering\n")
198 cat("Creating new feature: sleep_efficiency = deep_sleep_percentage + rem_percentage\n")
199 data$sleep_efficiency <- data$deep_sleep_percentage + data$rem_percentage
200 cat("\nSummary of sleep_efficiency:\n")
201 print(summary(data$sleep_efficiency))
202 |
203
```

OUTPUT

```
Task9: Feature Engineering
> cat("Creating new feature: sleep_efficiency = deep_sleep_percentage + rem_percentage\n")
Creating new feature: sleep_efficiency = deep_sleep_percentage + rem_percentage
> data$sleep_efficiency <- data$deep_sleep_percentage + data$rem_percentage
> cat("\nSummary of sleep_efficiency:\n")

Summary of sleep_efficiency:
> print(summary(data$sleep_efficiency))
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  16.7   36.6   40.6   40.5   44.5   60.0
> |
```

Output Description: The output presents the summary statistics of the newly created `sleep_efficiency` feature, including its minimum, maximum, mean, and quartile values.

Task 10: Class Balancing with Under- and Over-Sampling

Description: Display the original class distribution and perform both under-sampling and over-sampling to create balanced datasets for all target classes.

CODE

```
204
205 cat("\n Task 10: Class Balancing (Under/Over-Sampling) \n")
206 cat("Original class distribution\n")
207 class_counts <- table(data$sleep_disorder_risk)
208 print(class_counts)
209
210 cat("\n Under-sampling \n")
211 min_count <- min(class_counts)
212 cat("Minimum class count:", min_count, "\n")
213 set.seed(42)
214 under_idx <- unlist(lapply(names(class_counts), function(cl) {
215   return(sample(which(data$sleep_disorder_risk == cl), min_count))
216 })))
217 data_under <- data[under_idx, ]
218 cat("Under-sampled distribution:\n")
219 print(table(data_under$sleep_disorder_risk))
220
221 cat("\n Over-sampling \n")
222 max_count <- max(class_counts)
223 cat("Maximum class count:", max_count, "\n")
224 set.seed(42)
225 over_idx <- unlist(lapply(names(class_counts), function(cl) {
226   return(sample(which(data$sleep_disorder_risk == cl), max_count, replace = TRUE))
227 })))
228 data_over <- data[over_idx, ]
229 cat("Over-sampled distribution:\n")
230 print(table(data_over$sleep_disorder_risk))
231
232
```

OUTPUT

```
Task 10: Class Balancing (Under/Over-Sampling)
> cat("Original class distribution\n")
Original class distribution
> class_counts <- table(data$sleep_disorder_risk)
> print(class_counts)

Healthy    Mild Moderate    Severe
 54154     33477      8298     4066
>
> cat("\n Under-sampling \n")

Under-sampling
> min_count <- min(class_counts)
> cat("Minimum class count:", min_count, "\n")
Minimum class count: 4066
> set.seed(42)
> under_idx <- unlist(lapply(names(class_counts), function(cl) {
+   return(sample(which(data$sleep_disorder_risk == cl), min_count))
+ })))
> data_under <- data[under_idx, ]
> cat("Under-sampled distribution:\n")
Under-sampled distribution:
> print(table(data_under$sleep_disorder_risk))

Healthy    Mild Moderate    Severe
   4066     4066     4066     4066
>
> cat("\n Over-sampling \n")

Over-sampling
> max_count <- max(class_counts)
> cat("Maximum class count:", max_count, "\n")
Maximum class count: 54154
> set.seed(42)
> over_idx <- unlist(lapply(names(class_counts), function(cl) {
+   return(sample(which(data$sleep_disorder_risk == cl), max_count, replace = TRUE))
+ })))
> data_over <- data[over_idx, ]
> cat("Over-sampled distribution:\n")
Over-sampled distribution:
> print(table(data_over$sleep_disorder_risk))

Healthy    Mild Moderate    Severe
 54154     54154     54154     54154
>
```

Output Description: The output compares the original class distribution with the distributions after under-sampling and over-sampling, showing that all classes become balanced.

Task 11: Training and Testing Split

Description: Split the cleaned dataset into training (70%) and testing (30%) subsets using random sampling, so that a model can be trained and evaluated fairly on unseen data.

CODE

```
232
233
234 cat("\n Task11: Train/Test Split (70/30) \n")
235 set.seed(42)
236 train_idx <- sample(1:nrow(data), size = round(0.7 * nrow(data)))
237 train_data <- data[train_idx, ]
238 test_data <- data[-train_idx, ]
239 cat("Total rows:", nrow(data), "\n")
240 cat("Training rows:", nrow(train_data), "\n")
241 cat("Testing rows:", nrow(test_data), "\n")
242 cat("Train percentage:", round(nrow(train_data) / nrow(data) * 100, 1), "%\n")
243 cat("Test percentage:", round(nrow(test_data) / nrow(data) * 100, 1), "%\n")
244
```

OUTPUT

```
Task11: Train/Test Split (70/30)
> set.seed(42)
> train_idx <- sample(1:nrow(data), size = round(0.7 * nrow(data)))
> train_data <- data[train_idx, ]
> test_data <- data[-train_idx, ]
> cat("Total rows:", nrow(data), "\n")
Total rows: 99995
> cat("Training rows:", nrow(train_data), "\n")
Training rows: 69996
> cat("Testing rows:", nrow(test_data), "\n")
Testing rows: 29999
> cat("Train percentage:", round(nrow(train_data) / nrow(data) * 100, 1), "%\n")
Train percentage: 70 %
> cat("Test percentage:", round(nrow(test_data) / nrow(data) * 100, 1), "%\n")
Test percentage: 30 %
> |
```

Output Description: The output confirms that the dataset was successfully split into 69,996 (70%) training records and 29,999 (30%) testing records. This 70:30 split ensures that sufficient data is available for model training while reserving an independent testing set for evaluating model performance.

Task 12: Mean vs Median Imputation Comparison

Description: Apply both mean and median imputation to the selected numerical attribute, compare the standard deviation, mean, and median after imputation, and check the remaining missing values.

CODE

```
246
247 cat("\n Task12: Mean vs Median Imputation Comparison \n")
248 cat("Comparing mean and median imputation for stress_score\n\n")
249 cat("Missing values in stress_score:", sum(is.na(data$stress_score)), "\n\n")
250
251 sd_before <- sd(data$stress_score, na.rm = TRUE)
252 cat("Standard deviation BEFORE imputation:", round(sd_before, 4), "\n\n")
253
254 stress_mean_imputed <- data$stress_score
255 stress_mean_imputed[is.na(stress_mean_imputed)] <- mean(stress_mean_imputed, na.rm = TRUE)
256 sd_mean <- sd(stress_mean_imputed)
257
258 stress_median_imputed <- data$stress_score
259 stress_median_imputed[is.na(stress_median_imputed)] <- median(stress_median_imputed, na.rm = TRUE)
260 sd_median <- sd(stress_median_imputed)
261
262 cat("SD after MEAN imputation: ", round(sd_mean, 4), "\n")
263 cat("SD after MEDIAN imputation: ", round(sd_median, 4), "\n\n")
264
265 cat("Mean of original: ", round(mean(data$stress_score, na.rm = TRUE), 4), "\n")
266 cat("Mean after mean imputation: ", round(mean(stress_mean_imputed), 4), "\n")
267 cat("Mean after median imputation: ", round(mean(stress_median_imputed), 4), "\n\n")
268
269 cat("Median of original: ", round(median(data$stress_score, na.rm = TRUE), 4), "\n")
270 cat("Median after mean imputation: ", round(median(stress_mean_imputed), 4), "\n")
271 cat("Median after median imputation: ", round(median(stress_median_imputed), 4), "\n\n")
272
273 cat("Conclusion: Median imputation preserves the original SD and distribution\n")
274 cat("better because it is less sensitive to outliers than the mean.\n")
275
276 data$stress_score <- stress_median_imputed
277 data$stress_score_minmax <- min_max_normalize(data$stress_score)
278
279 cat("\nFinal missing values check:\n")
280 print(colSums(is.na(data)))
281
```

OUTPUT

```
Task12: Mean vs Median Imputation Comparison
> cat("Comparing mean and median imputation for stress_score\n\n")
Comparing mean and median imputation for stress_score

> cat("Missing values in stress_score:", sum(is.na(data$stress_score)), "\n\n")
Missing values in stress_score: 8109

>
> sd_before <- sd(data$stress_score, na.rm = TRUE)
> cat("Standard deviation BEFORE imputation:", round(sd_before, 4), "\n\n")
Standard deviation BEFORE imputation: 21.3544

> cat("SD after MEAN imputation: ", round(sd_mean, 4), "\n")
SD after MEAN imputation: 20.4702
> cat("SD after MEDIAN imputation: ", round(sd_median, 4), "\n\n")
SD after MEDIAN imputation: 20.486

> cat("Mean of original: ", round(mean(data$stress_score, na.rm = TRUE), 4), "\n")
Mean of original: 8.841
> cat("Mean after mean imputation: ", round(mean(stress_mean_imputed), 4), "\n")
Mean after mean imputation: 8.841
> cat("Mean after median imputation: ", round(mean(stress_median_imputed), 4), "\n\n")
Mean after median imputation: 8.6025

> cat("Median of original: ", round(median(data$stress_score, na.rm = TRUE), 4), "\n")
Median of original: 5.9
> cat("Median after mean imputation: ", round(median(stress_mean_imputed), 4), "\n")
Median after mean imputation: 6
> cat("Median after median imputation: ", round(median(stress_median_imputed), 4), "\n\n")
Median after median imputation: 5.9

> cat("Conclusion: Median imputation preserves the original SD and distribution\n")
Conclusion: Median imputation preserves the original SD and distribution
> cat("better because it is less sensitive to outliers than the mean.\n")
better because it is less sensitive to outliers than the mean.
>
```

```
Final missing values check:
> print(colSums(is.na(data)))
      person_id      age      gender
      0          0          0
      occupation      bmi      country
      0          0          0
      sleep_duration_hrs sleep_quality_score      rem_percentage
      0          0          0
      deep_sleep_percentage      sleep_latency_mins      wake_episodes_per_night
      0          0          0
      caffeine_mg_before_bed      alcohol_units_before_bed      screen_time_before_bed_mins
      0          0          0
      exercise_day      steps_that_day      nap_duration_mins
      0          0          0
      stress_score      work_hours_that_day      chronotype
      0          0          0
      mental_health_condition      heart_rate_resting_bpm      sleep_aid_used
      0          0          0
      shift_work      room_temperature_celsius      weekend_sleep_diff_hrs
      0          0          0
      season      day_type      cognitive_performance_score
      0          0          0
      sleep_disorder_risk      felt_rested      age_group
      0          0          0
      gender_num      stress_score_minmax      sleep_efficiency
      0          0          0
```

Output Description: The output compares the statistical values obtained after mean and median imputation and confirms that all missing values have been successfully handled.

Task 13: Descriptive Statistics by Target Class

Description: Group the dataset by the target class and calculate the mean, standard deviation, and median of important numerical attributes for each class.

CODE

```
284 cat("\n Task13: Descriptive Statistics by Target Class \n")
285 cat("Descriptive statistics grouped by sleep_disorder_risk:\n\n")
286 classes <- unique(data$sleep_disorder_risk)
287 for (cl in sort(classes)) {
288   sub <- data[data$sleep_disorder_risk == cl, ]
289   cat("Class:", cl, "(n =", nrow(sub), ") \n")
290   cat("  sleep_duration_hrs -> Mean:", round(mean(sub$sleep_duration_hrs), 2),
291       " SD:", round(sd(sub$sleep_duration_hrs), 2),
292       " Median:", round(median(sub$sleep_duration_hrs), 2), "\n")
293   cat("  stress_score       -> Mean:", round(mean(sub$stress_score), 2),
294       " SD:", round(sd(sub$stress_score), 2),
295       " Median:", round(median(sub$stress_score), 2), "\n")
296   cat("    bmi              -> Mean:", round(mean(sub$bmi), 2),
297       " SD:", round(sd(sub$bmi), 2),
298       " Median:", round(median(sub$bmi), 2), "\n\n")
299 }
300
```

OUTPUT

```
--- Class: Healthy (n = 54154 ) ---
sleep_duration_hrs -> Mean: 6.92 SD: 1.3 Median: 6.71
stress_score       -> Mean: 8.01 SD: 20.7 Median: 5.4
bmi                -> Mean: 25.87 SD: 6.1 Median: 25.9

--- Class: Mild (n = 33477 ) ---
sleep_duration_hrs -> Mean: 6.16 SD: 1.5 Median: 6.04
stress_score       -> Mean: 9.09 SD: 20.19 Median: 6.3
bmi                -> Mean: 27.32 SD: 6.18 Median: 26.7

--- Class: Moderate (n = 8298 ) ---
sleep_duration_hrs -> Mean: 5.58 SD: 1.48 Median: 5.31
stress_score       -> Mean: 9.62 SD: 19.9 Median: 7.1
bmi                -> Mean: 28.15 SD: 6.14 Median: 27.8

--- Class: Severe (n = 4066 ) ---
sleep_duration_hrs -> Mean: 5.09 SD: 1.47 Median: 4.94
stress_score       -> Mean: 10.38 SD: 20.98 Median: 7.4
bmi                -> Mean: 28.96 SD: 5.95 Median: 28.9
```

Output Description: The output displays descriptive statistics for each target class separately, allowing comparison of numerical attributes across all sleep disorder categories.

Task 14: Comparing Averages Across Two Categories

Description: Calculate the average sleep duration for shift workers and non-shift workers using `tapply()` and compare the two groups with a two-sample t-test.

CODE

```
300
301 cat("\n Task14: Comparing Averages Across Two Categories \n")
302 cat("Comparing average sleep_duration_hrs across shift_work (0=No, 1=Yes)\n\n")
303
304 avg_by_shift <- tapply(data$sleep_duration_hrs, data$shift_work,| mean)
305 cat("Average sleep duration by shift work:\n")
306 print(round(avg_by_shift, 4))
307
308 cat("\nT-test result:\n")
309 t_result <- t.test(sleep_duration_hrs ~ shift_work, data = data)
310 print(t_result)
311
312
```

OUTPUT

```
Task14: Comparing Averages Across Two Categories
> cat("Comparing average sleep_duration_hrs across shift_work (0=No, 1=Yes)\n\n")
Comparing average sleep_duration_hrs across shift_work (0=No, 1=Yes)

>
> avg_by_shift <- tapply(data$sleep_duration_hrs, data$shift_work, mean)
> cat("Average sleep duration by shift work:\n")
Average sleep duration by shift work:
> print(round(avg_by_shift, 4))
      0      1
6.5576 5.6471
>
> cat("\nT-test result:\n")

T-test result:
> t_result <- t.test(sleep_duration_hrs ~ shift_work, data = data)
> print(t_result)

      Welch Two Sample t-test

data:  sleep_duration_hrs by shift_work
t = 53.709, df = 10008, p-value < 2.2e-16
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
95 percent confidence interval:
 0.8772542 0.9437139
sample estimates:
mean in group 0 mean in group 1
 6.557616      5.647132

> |
```

Output Description: The output shows the average sleep duration for both groups along with the complete t-test result, including the p-value and confidence interval.

Task 15: Variability Comparison Across Categories

Description: Compare the variability of stress_score across different chronotype groups by calculating the IQR, standard deviation, variance, and range.

CODE

```
313
314 cat("\n Task15: Variability Comparison Across Categories \n")
315 cat("Comparing variability of stress_score across chronotype categories\n\n")
316
317 chrono_levels <- unique(data$chronotype)
318 for (ch in sort(chrono_levels)) {
319   sub <- data[data$chronotype == ch, ]
320   cat("Chronotype:", ch, "(n =", nrow(sub), ") \n")
321   cat("  IQR:           ", round(IQR(sub$stress_score), 3), "\n")
322   cat("  Standard Deviation:", round(sd(sub$stress_score), 3), "\n")
323   cat("  Variance:         ", round(var(sub$stress_score), 3), "\n")
324   cat("  Range:           ", round(diff(range(sub$stress_score)), 3), "\n\n")
325 }
326
```

OUTPUT

```
Task15: Variability Comparison Across Categories
> cat("Comparing variability of stress_score across chronotype categories\n\n")
Comparing variability of stress_score across chronotype categories

>
> chrono_levels <- unique(data$chronotype)
> for (ch in sort(chrono_levels)) {
+   sub <- data[data$chronotype == ch, ]
+   cat("Chronotype:", ch, "(n =", nrow(sub), ") \n")
+   cat("  IQR:           ", round(IQR(sub$stress_score), 3), "\n")
+   cat("  Standard Deviation:", round(sd(sub$stress_score), 3), "\n")
+   cat("  Variance:         ", round(var(sub$stress_score), 3), "\n")
+   cat("  Range:           ", round(diff(range(sub$stress_score)), 3), "\n\n")
+ }
Chronotype: Evening (n = 29643 )
  IQR:           1.9
  Standard Deviation: 20.695
  Variance:         428.297
  Range:           179

Chronotype: Morning (n = 24959 )
  IQR:           1.9
  Standard Deviation: 19.981
  Variance:         399.248
  Range:           179

Chronotype: Neutral (n = 45393 )
  IQR:           1.9
  Standard Deviation: 20.622
  Variance:         425.278
  Range:           179
```

Output Description: The output presents the variability measures for each chronotype category, making it easy to compare the spread of stress scores among the groups.

PROJECT CODE

```
cat("\n Task1: Exploratory Data
Analysis \n")
data <-
read.csv("C:\\Users\\Farhan\\Downloa
ds\\archive\\sleep_health_dataset_mo
dified.csv",
        header = TRUE, sep
= ",",
        stringsAsFactors =
FALSE, na.strings = c("NA", ""))

raw_data <- data

cat("Structure of the dataset: \n")
str(data)

cat("\n First 6 rows: \n")
print(head(data))

cat("\n Dataset size: \n")
cat("Rows:", nrow(data), "\n")
cat("Columns:", ncol(data), "\n")

cat("\nColumn names: \n")
print(names(data))

cat("\n Summary of the dataset: \n")
print(summary(data))

cat("\n Target variable
(sleep_disorder_risk) distribution:
\n")
print(table(data$sleep_disorder_risk
))
cat("Proportions (%): \n")
print(round(prop.table(table(data$sl
eep_disorder_risk)) * 100, 2))

cat("\n Task8: Invalid Data
Detection and Handling \n")
cat("(Performed early so numeric
columns are clean for subsequent
tasks) \n\n")

cat("Cleaning non-numeric values in
bmi \n")
invalid_bmi_text <- sum(grepl("[^0-
9.\\-]", data$bmi), na.rm = TRUE)
cat("Non-numeric bmi values found:",
invalid_bmi_text, "\n")
data$bmi <-
suppressWarnings(as.numeric(data$bmi
))
cat("Converted bmi to numeric (non-
numeric values become NA) \n")
```

```
cat("\n Cleaning non-numeric values
in sleep_duration_hrs \n")
invalid_text_idx <- grepl("[^0-9.\\-
]", data$sleep_duration_hrs)
cat("Non-numeric sleep_duration_hrs
values found:",
sum(invalid_text_idx, na.rm = TRUE),
"\n")
cat("Examples:",
head(data$sleep_duration_hrs[invalid
_text_idx], 5), "\n")
data$sleep_duration_hrs <-
suppressWarnings(as.numeric(data$sl
eep_duration_hrs))
cat("Converted to numeric (non-
numeric values become NA) \n")

cat("\n Handling negative age values
\n")
invalid_age_count <- sum(data$age <
0, na.rm = TRUE)
cat("Negative age values found:",
invalid_age_count, "\n")
data$age[data$age < 0] <- NA
med_age <- round(median(data$age,
na.rm = TRUE))
data$age[is.na(data$age)] <- med_age
cat("Replaced with median age:",
med_age, "\n")

cat("\n Handling negative
sleep_duration_hrs \n")
neg_sleep <-
sum(data$sleep_duration_hrs < 0,
na.rm = TRUE)
cat("Negative sleep_duration values
found:", neg_sleep, "\n")
data$sleep_duration_hrs[data$sleep_d
uration_hrs < 0] <- NA
cat("Set negative sleep values to NA
(will be imputed in Task 2)\n")

cat("\n Handling
heart_rate_resting_bpm <= 30 \n")
low_hr <-
sum(data$heart_rate_resting_bpm <=
30, na.rm = TRUE)
cat("Heart rate <= 30 values
found:", low_hr, "\n")
data$heart_rate_resting_bpm[data$hea
rt_rate_resting_bpm <= 30] <- NA
med_hr <-
round(median(data$heart_rate_resting
_bpm, na.rm = TRUE))
```

```

data$heart_rate_resting_bpm[is.na(data$heart_rate_resting_bpm)] <-
med_hr
cat("Replaced with median heart
rate:", med_hr, "\n")

cat("\n Handling invalid gender
values \n")
cat("Gender distribution before
cleaning:\n")
print(table(data$gender))
data$gender[!(data$gender %in%
c("Male", "Female", "Other"))] <-
"Other"
cat("Gender distribution after
cleaning:\n")
print(table(data$gender))

cat("\n Task2: Missing Value
Treatment \n")
cat("Missing values BEFORE
treatment:\n")
print(colSums(is.na(data)))
cat("Total missing values:",
sum(is.na(data)), "\n\n")

cat(" Imputing numerical columns
with median \n")
data$bmi[is.na(data$bmi)] <-
median(data$bmi, na.rm = TRUE)
cat("bmi imputed with median\n")

data$sleep_duration_hrs[is.na(data$sleep_duration_hrs)] <-
median(data$sleep_duration_hrs,
na.rm = TRUE)
cat("sleep_duration_hrs imputed with
median\n")

data$cognitive_performance_score[is.na(data$cognitive_performance_score)] <-
median(data$cognitive_performance_score, na.rm = TRUE)
cat("cognitive_performance_score
imputed with median\n")

cat("\n Imputing categorical columns
with mode \n")
get_mode <- function(v) {
  uniqv <- unique(v[!is.na(v)])

  return(uniqv[which.max(tabulate(match(v, uniqv)))] )
}

data$occupation[is.na(data$occupation)] <- get_mode(data$occupation)
cat("occupation imputed with mode:",
get_mode(raw_data$occupation), "\n")

data$mental_health_condition[is.na(data$mental_health_condition)] <-
get_mode(data$mental_health_condition)
cat("mental_health_condition imputed
with mode:",
get_mode(raw_data$mental_health_condition), "\n")

cat("\nMissing values AFTER
treatment (stress_score handled in
Task 12):\n")
print(colSums(is.na(data)))
cat("Total missing values:",
sum(is.na(data)), "\n\n")

cat("\n Task3: Outlier Detection and
Handling \n")
detect_outliers_iqr <- function(x) {
  q1 <- quantile(x, 0.25, na.rm =
TRUE)
  q3 <- quantile(x, 0.75, na.rm =
TRUE)
  iqr_val <- q3 - q1
  return(which(x < q1 - 1.5 *
iqr_val | x > q3 + 1.5 * iqr_val))
}

cap_outliers_iqr <- function(x) {
  q1 <- quantile(x, 0.25, na.rm =
TRUE)
  q3 <- quantile(x, 0.75, na.rm =
TRUE)
  iqr_val <- q3 - q1
  lower <- q1 - 1.5 * iqr_val
  upper <- q3 + 1.5 * iqr_val
  x[x < lower] <- lower
  x[x > upper] <- upper
  return(x)
}

cat(" steps_that_day \n")
outliers_steps <-
detect_outliers_iqr(data$steps_that_day)
cat("Outliers before capping:",
length(outliers_steps), "\n")
data$steps_that_day <-
cap_outliers_iqr(data$steps_that_day)
)

```

```
cat("Outliers after capping:",
length(detect_outliers_iqr(data$steps_that_day)), "\n")
```

```
cat("\n screen_time_before_bed_mins
\n")
outliers_screen <-
detect_outliers_iqr(data$screen_time
_before_bed_mins)
cat("Outliers before capping:",
length(outliers_screen), "\n")
data$screen_time_before_bed_mins <-
cap_outliers_iqr(data$screen_time_b
efore_bed_mins)
cat("Outliers after capping:",
length(detect_outliers_iqr(data$scree
n_time_before_bed_mins)), "\n")
```

```
cat("\n Task4: Numeric to
Categorical and Categorical to
Numeric \n")
cat(" Numeric to Categorical: age ->
age_group \n")
data$age_group <- cut(data$age,
breaks = c(17,
30, 50, 100),
labels =
c("Young", "Middle-aged", "Senior"))
cat("age_group distribution:\n")
print(table(data$age_group))
```

```
cat("\n Categorical to Numeric:
gender -> gender_num \n")
data$gender_num <-
as.numeric(factor(data$gender,
```

```
levels = c("Female", "Male",
"Other")))
cat("Mapping: Female=1, Male=2,
Other=3\n")
print(table(data$gender,
data$gender_num))
```

```
cat("\n Task5: Normalization \n")
min_max_normalize <- function(x) {
return((x - min(x, na.rm = TRUE))
/ (max(x, na.rm = TRUE) - min(x,
na.rm = TRUE)))
}
```

```
cat("stress_score BEFORE
normalization:\n")
print(summary(data$stress_score))
```

```
data$stress_score_minmax <-
min_max_normalize(data$stress_score)
cat("\nstress_score AFTER Min-Max
normalization:\n")
print(summary(data$stress_score_minm
ax))
```

```
cat("\n Task 6: Duplicate Detection
and Removal \n")
dup_count <- sum(duplicated(data[, -
which(names(data) == "person_id")]))
cat("Total rows:", nrow(data), "\n")
cat("Duplicate rows found:",
dup_count, "\n")
data <- data[!duplicated(data[, -
which(names(data) == "person_id")]),
]
cat("Rows after removing
duplicates:", nrow(data), "\n")
```

```
cat("\n Task 7: Data Filtering \n")
cat("Filtering data to keep only top
5 countries by frequency\n\n")
cat("Country distribution:\n")
print(sort(table(data$country),
decreasing = TRUE))
```

```
top5 <-
names(sort(table(data$country),
decreasing = TRUE))[1:5]
cat("\nTop 5 countries:", top5,
"\n")
```

```
data_filtered <- data[data$country
%in% top5, ]
cat("Rows before filtering:",
nrow(data), "\n")
cat("Rows after filtering:",
nrow(data_filtered), "\n")
```

```
cat("\n Task9: Feature Engineering
\n")
cat("Creating new feature:
sleep_efficiency =
deep_sleep_percentage +
rem_percentage\n")
data$sleep_efficiency <-
data$deep_sleep_percentage +
data$rem_percentage
cat("\nSummary of
sleep_efficiency:\n")
```

```

print(summary(data$sleep_efficiency)
)

cat("\n Task 10: Class Balancing
(Under/Over-Sampling) \n")
cat("Original class distribution\n")
class_counts <-
table(data$sleep_disorder_risk)
print(class_counts)

cat("\n Under-sampling \n")
min_count <- min(class_counts)
cat("Minimum class count:",
min_count, "\n")
set.seed(42)
under_idx <-
unlist(lapply(names(class_counts),
function(cl) {

return(sample(which(data$sleep_disor
der_risk == cl), min_count))
}))
data_under <- data[under_idx, ]
cat("Under-sampled distribution:\n")
print(table(data_under$sleep_disorde
r_risk))

cat("\n Over-sampling \n")
max_count <- max(class_counts)
cat("Maximum class count:",
max_count, "\n")
set.seed(42)
over_idx <-
unlist(lapply(names(class_counts),
function(cl) {

return(sample(which(data$sleep_disor
der_risk == cl), max_count, replace
= TRUE))
}))
data_over <- data[over_idx, ]
cat("Over-sampled distribution:\n")
print(table(data_over$sleep_disorder
_risk))

cat("\n Task11: Train/Test Split
(70/30) \n")
set.seed(42)
train_idx <- sample(1:nrow(data),
size = round(0.7 * nrow(data)))
train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]
cat("Total rows:", nrow(data), "\n")
cat("Training rows:",
nrow(train_data), "\n")

cat("Testing rows:",
nrow(test_data), "\n")
cat("Train percentage:",
round(nrow(train_data) / nrow(data)
* 100, 1), "%\n")
cat("Test percentage:",
round(nrow(test_data) / nrow(data) *
100, 1), "%\n")

cat("\n Task12: Mean vs Median
Imputation Comparison \n")
cat("Comparing mean and median
imputation for stress_score\n\n")
cat("Missing values in
stress_score:",
sum(is.na(data$stress_score)),
"\n\n")

sd_before <- sd(data$stress_score,
na.rm = TRUE)
cat("Standard deviation BEFORE
imputation:", round(sd_before, 4),
"\n\n")

stress_mean_imputed <-
data$stress_score
stress_mean_imputed[is.na(stress_mea
n_imputed)] <-
mean(stress_mean_imputed, na.rm =
TRUE)
sd_mean <- sd(stress_mean_imputed)

stress_median_imputed <-
data$stress_score
stress_median_imputed[is.na(stress_m
edian_imputed)] <-
median(stress_median_imputed, na.rm
= TRUE)
sd_median <-
sd(stress_median_imputed)

cat("SD after MEAN imputation: ",
round(sd_mean, 4), "\n")
cat("SD after MEDIAN imputation: ",
round(sd_median, 4), "\n\n")

cat("Mean of original:
", round(mean(data$stress_score,
na.rm = TRUE), 4), "\n")
cat("Mean after mean imputation:
", round(mean(stress_mean_imputed),
4), "\n")
cat("Mean after median imputation:
",
round(mean(stress_median_imputed),
4), "\n\n")

```

```

cat("Median of original:
", round(median(data$stress_score,
na.rm = TRUE), 4), "\n")
cat("Median after mean imputation:
",
round(median(stress_mean_imputed),
4), "\n")
cat("Median after median imputation:
",
round(median(stress_median_imputed),
4), "\n\n")

cat("Conclusion: Median imputation
preserves the original SD and
distribution\n")
cat("better because it is less
sensitive to outliers than the
mean.\n")

data$stress_score <-
stress_median_imputed
data$stress_score_minmax <-
min_max_normalize(data$stress_score)

cat("\nFinal missing values
check:\n")
print(colSums(is.na(data)))

cat("\n Task13: Descriptive
Statistics by Target Class \n")
cat("Descriptive statistics grouped
by sleep_disorder_risk:\n\n")
classes <-
unique(data$sleep_disorder_risk)
for (cl in sort(classes)) {
  sub <-
data[data$sleep_disorder_risk == cl,
]
  cat("Class:", cl, "(n =",
nrow(sub), ") \n")
  cat("  sleep_duration_hrs ->
Mean:",
round(mean(sub$sleep_duration_hrs),
2),
" SD:",
round(sd(sub$sleep_duration_hrs),
2),
" Median:",
round(median(sub$sleep_duration_hrs)
, 2), "\n")
  cat("  stress_score ->
Mean:",
round(mean(sub$stress_score), 2),
" SD:",
round(sd(sub$stress_score), 2),
" Median:",
round(median(sub$stress_score), 2),
"\n")
}

cat("\n Task14: Comparing Averages
Across Two Categories \n")
cat("Comparing average
sleep_duration_hrs across shift_work
(0=No, 1=Yes)\n\n")

avg_by_shift <-
tapply(data$sleep_duration_hrs,
data$shift_work, mean)
cat("Average sleep duration by shift
work:\n")
print(round(avg_by_shift, 4))

cat("\nT-test result:\n")
t_result <-
t.test(sleep_duration_hrs ~
shift_work, data = data)
print(t_result)

cat("\n Task15: Variability
Comparison Across Categories \n")
cat("Comparing variability of
stress_score across chronotype
categories\n\n")

chrono_levels <-
unique(data$chronotype)
for (ch in sort(chrono_levels)) {
  sub <- data[data$chronotype == ch,
]
  cat("Chronotype:", ch, "(n =",
nrow(sub), ") \n")
  cat("  IQR: ",
round(IQR(sub$stress_score), 3),
"\n")
  cat("  Standard Deviation:",
round(sd(sub$stress_score), 3),
"\n")
  cat("  Variance: ",
round(var(sub$stress_score), 3),
"\n")
  cat("  Range: ",
round(diff(range(sub$stress_score)),
3), "\n\n")
}

```